

程序化无限像素世界开发计划

1. 项目概述

1.1 项目名称

暂定英文名：

Infinite Frontier

中文名：

无尽边境

1.2 项目定位

开发一款完全本地运行的 2D 俯视角像素生存探索游戏。

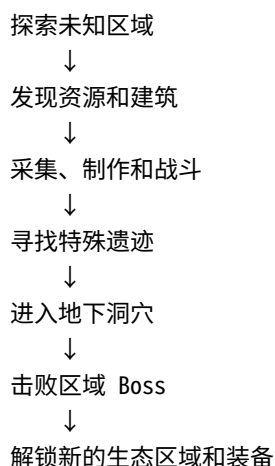
游戏不使用大语言模型，不连接任何云端 API。世界中的地形、生态群系、河流、道路、村庄、遗迹、洞穴、资源和敌人，全部通过传统程序化生成算法产生。

玩家可以不断向世界任意方向探索。地图在玩家接近边缘时自动生成新区块，远离玩家的区块自动卸载，从而形成理论上无限大的世界。

1.3 第一版核心体验

玩家出生在随机世界中，通过探索和收集逐渐生存下来。

核心流程：



1.4 项目特点

特点	说明
无限世界	根据世界种子按区块动态生成
稳定复现	相同种子生成完全相同的基础世界
多生态群系	森林、平原、沙漠、雪原、沼泽等
动态加载	只保留玩家附近区块，控制内存占用
世界持久化	玩家破坏、建造和开启过的内容永久保存
程序化建筑	村庄、遗迹、营地和地牢自动生成
地表与地下	地表世界和洞穴世界分层运行
生存探索	资源、制作、战斗、天气和昼夜系统
可扩展结构	后续可加入联机、模组和更多世界层

2. 开发目标

2.1 核心目标

第一版必须实现：

1. 创建和加载世界。
2. 输入或随机生成世界种子。
3. 根据玩家位置动态加载和卸载区块。
4. 生成连续且自然的地形。
5. 根据温度和湿度划分生态群系。
6. 生成树木、石头、矿物和植物。
7. 生成河流、道路、村庄和遗迹。
8. 支持角色移动、采集、背包和基础制作。
9. 支持基础敌人和战斗。
10. 支持昼夜与简单天气。
11. 支持地表与洞穴两个世界层。
12. 保存玩家对世界的所有永久修改。
13. 在普通电脑上稳定运行。
14. 最终打包为 Windows EXE。

2.2 项目学习内容

方向	主要内容
游戏引擎	Godot 节点、场景、资源、信号
程序化生成	噪声、种子随机、区块生成
地图系统	TileMapLayer、分层地图、坐标转换

方向	主要内容
性能管理	区块加载、对象池、后台生成
算法	A*、元胞自动机、泊松圆盘采样
游戏系统	状态机、背包、制作、战斗
数据存储	区块差异保存、角色存档
软件架构	模块解耦、事件总线、数据驱动
测试	确定性测试、地图连通性测试
发布	Windows 打包、配置和错误日志

3. 项目范围

3.1 第一版必须包含

模块	第一版内容
世界系统	世界种子、区块坐标、动态加载
地形生成	海洋、陆地、海岸、高低地
生态群系	平原、森林、沙漠、雪原、沼泽
地表资源	树木、石头、草、浆果、矿石
建筑系统	营地、废墟、村庄、地牢入口
道路系统	村庄和建筑之间生成道路
地下洞穴	洞穴、矿物、地下敌人
玩家系统	移动、生命、体力、交互
采集系统	砍树、采矿、采集植物
背包系统	物品、堆叠、快捷栏
制作系统	工具、武器、基础建筑材料
战斗系统	近战、受伤、死亡、掉落
敌人系统	3~5 种基础敌人
时间系统	昼夜循环
天气系统	晴天、下雨、下雪
存档系统	世界、角色和区块修改保存
基础 UI	主菜单、背包、生命、地图信息

3.2 第一版暂不实现

以下内容不要在第一版加入：

- 在线联机
- 云存档
- 大语言模型
- NPC 自由对话
- 复杂农业
- 自动化工厂
- 复杂科技树
- 坐骑
- 船只和车辆
- 完整季节系统
- 大规模战争
- 复杂城市模拟
- Mod 编辑器
- Steam 创意工坊
- 3D 地图
- 真实光照模拟

这些内容可以在基础系统稳定后逐步增加。

4. 游戏视角与美术规格

4.1 游戏视角

第一版采用：

2D 俯视角
固定正交摄像机
像素风格
八方向移动

不采用横版的原因：

- 俯视角区块生成更简单。
- 河流、道路和村庄更容易表现。
- 地图可以向四个方向无限扩展。
- 敌人寻路和建筑摆放更容易。
- 地表与地下层可以独立生成。
- Codex 更容易完成高完成度版本。

4.2 像素规格

建议统一使用：

项目	推荐值
基础地块尺寸	16×16 像素
角色尺寸	16×24 或 24×24 像素
世界单元格	1 个地块
目标窗口	1280×720
内部渲染分辨率	640×360
缩放倍率	2 倍整数缩放
动画帧率	6~12 FPS
游戏目标帧率	60 FPS

必须启用像素对齐，避免摄像机移动时出现像素抖动和贴图模糊。

5. 技术选型

5.1 推荐技术栈

模块	技术
游戏引擎	Godot 4.x
编程语言	GDScript
地图渲染	TileMapLayer
地形噪声	FastNoiseLite
路径寻找	AStarGrid2D
数据资源	Godot Resource
配置数据	JSON 或自定义 Resource
存档数据	二进制文件或 SQLite
自动测试	GUT 或自定义测试场景
版本管理	Git
Windows 发布	Godot Windows Export

5.2 为什么优先使用 GDScript

- 与 Godot 结合紧密。
- 开发速度快。
- Codex 容易生成和修改。
- 调试时可以快速热重载。
- 第一版性能瓶颈不会主要来自脚本语言。

- 后续可以将重计算模块改成 C# 或 GDExtension。

5.3 数据存储建议

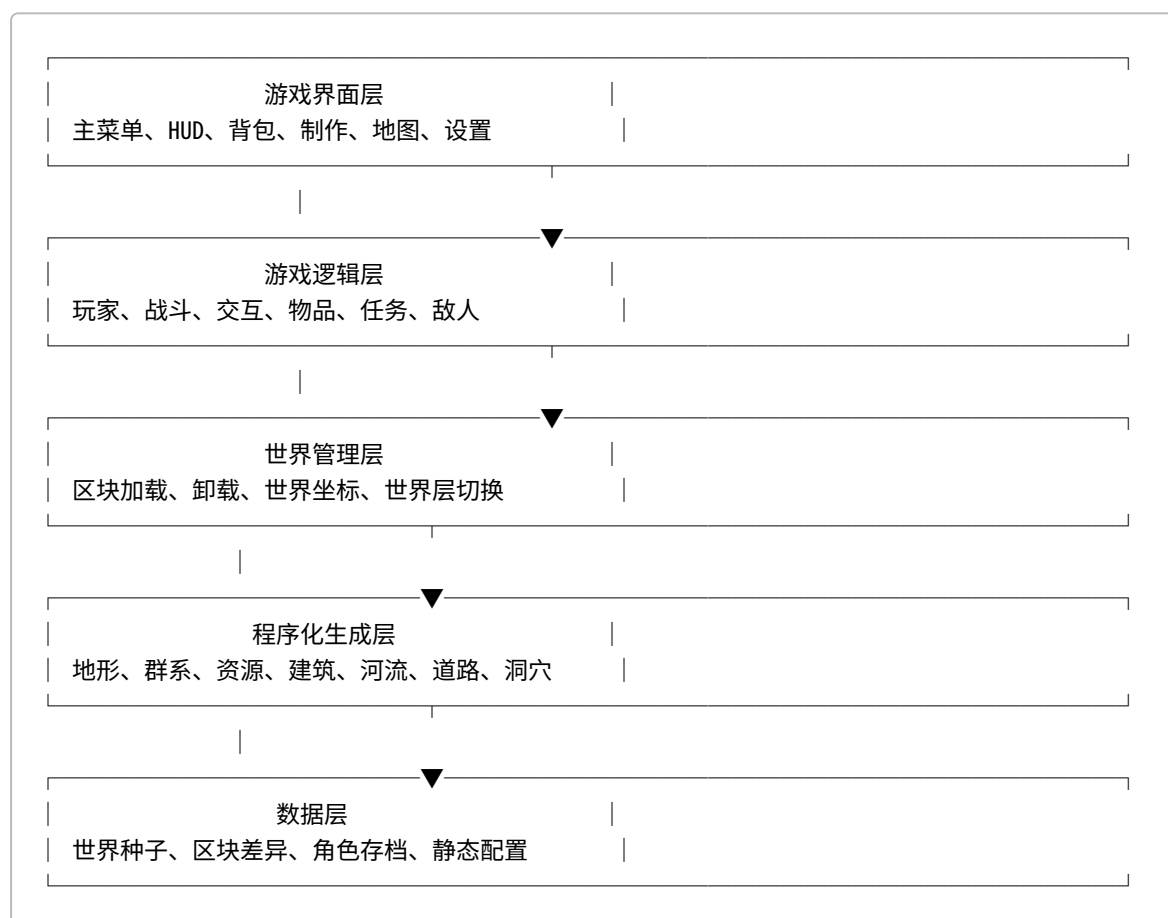
第一版采用：

世界元数据：JSON
玩家存档：JSON 或二进制
区块差异：压缩二进制文件
设置选项：ConfigFile
静态物品数据：Godot Resource

不要一开始就使用复杂数据库。

当区块数量较多、查询需求明显时，再考虑 SQLite。

6. 总体架构



6.1 设计原则

1. 世界生成模块不能直接操作玩家。

2. 玩家系统不能直接保存文件。
3. 区块生成结果必须由数据结构表示。
4. 地图渲染与区块数据分离。
5. 所有随机生成都必须基于固定种子。
6. 生成逻辑不依赖当前帧率。
7. 静态配置尽量数据驱动。
8. 不允许将所有系统写进一个主脚本。
9. 所有永久修改统一通过世界状态管理器记录。
10. 未加载区块不能保留大量节点。

7. 核心概念与坐标系统

7.1 世界坐标

整个世界使用整数地块坐标：

```
Tile Coordinate = (tile_x, tile_y)
```

例如：

```
(0, 0)      世界原点  
(120, -35)  原点右侧 120 格，上方 35 格
```

7.2 区块坐标

世界被划分为固定大小区块。

建议第一版：

```
每个区块：32 × 32 个地块
```

区块坐标计算：

```
chunk_x = floor(tile_x / 32)  
chunk_y = floor(tile_y / 32)
```

区块内坐标：

```
local_x = tile_x mod 32  
local_y = tile_y mod 32
```

必须正确处理负坐标，不能直接使用会产生错误结果的普通整数除法。

7.3 世界层

第一版至少包含两个世界层：

世界层	说明
Surface	地表世界
Underground	地下洞穴

区块唯一标识：

```
world_layer + chunk_x + chunk_y
```

示例：

```
surface_5_-2  
underground_5_-2
```

8. 区块系统

8.1 区块数据结构

```
ChunkData  
- chunk_position: Vector2i  
- world_layer: int  
- generation_version: int  
- base_tiles: PackedInt32Array  
- decoration_tiles: PackedInt32Array  
- biome_map: PackedByteArray  
- structures: Array  
- resource_nodes: Array  
- entities: Array  
- discovered: bool  
- modified: bool
```

8.2 区块状态

```
UNGENERATED  
GENERATING  
GENERATED  
LOADING  
ACTIVE  
SLEEPING
```


UNLOADING
SAVED

8.3 加载范围

建议初始配置：

范围	半径
完整活动区块	玩家周围 2 个区块
可见和预加载区块	玩家周围 3 个区块
仅保留数据区块	玩家周围 4 个区块
超出范围	保存并卸载

玩家周围最多完整激活：

$5 \times 5 = 25$ 个区块

预加载范围：

$7 \times 7 = 49$ 个区块

8.4 区块加载流程

```
graph TD
    A[玩家移动] --> B[计算玩家当前区块]
    B --> C[与上一次区块比较]
    C --> D[确定需要加载的新区块]
    D --> E[读取已有存档或程序生成]
    E --> F[创建地图和对象]
    F --> G[确定需要卸载的旧区块]
    G --> H[保存差异]
    H --> I[释放节点]
```

8.5 区块生成优先级

距离玩家越近，生成优先级越高。

推荐顺序：

1. 玩家即将进入的区块。
2. 玩家当前移动方向上的区块。
3. 相邻可见区块。
4. 其他预加载区块。

9. 世界种子与确定性生成

9.1 世界种子

每个世界拥有一个 64 位整数种子。

用户可以：

- 输入文字种子。
- 输入数字种子。
- 点击随机生成。
- 复制当前种子。

文字种子需要通过稳定哈希转换为整数。

不能使用可能因平台或运行版本变化而改变结果的默认哈希函数。

9.2 派生种子

不同系统不能共用同一随机序列。

应从主种子派生：

```
terrain_seed  
temperature_seed  
moisture_seed  
river_seed  
structure_seed  
resource_seed  
enemy_seed  
loot_seed  
cave_seed
```

派生过程必须稳定。

9.3 坐标随机

区块生成不依赖生成顺序。

正确做法：

```
局部随机种子 =  
世界种子  
+ 世界层  
+ 区块 X  
+ 区块 Y  
+ 生成类型
```

这样无论玩家从哪个方向接近，同一区块生成结果都相同。

9.4 生成版本

世界元数据中记录：

```
generation_version
```

生成算法改变后，旧世界仍使用原来的生成版本，避免游戏更新后已探索区域与新区块严重断裂。

10. 地形生成

10.1 基础地形参数

使用多个噪声图：

噪声图	用途
Elevation	海拔和海陆分布
Continentalness	大陆与海洋结构
Erosion	平原、丘陵和崎岖度
Temperature	温度
Moisture	湿度
Detail	地形细节
Variation	打破规则边界

10.2 地形判定

示例规则：

海拔 < 0.28	深海
海拔 < 0.36	浅海
海拔 < 0.40	沙滩
海拔 < 0.68	普通陆地
海拔 < 0.82	丘陵
海拔 $\geq$ 0.82	山地

具体数值应通过调试面板实时调整。

10.3 多尺度噪声

不能只使用单层噪声。

建议组合：

大陆噪声：低频、大尺度
地形噪声：中频
细节噪声：高频、小权重

最终海拔：

```
elevation =  
continentalness  $\times$  0.60  
+ terrain_noise  $\times$  0.30  
+ detail_noise  $\times$  0.10
```

10.4 海岸平滑

直接阈值划分容易产生锯齿海岸。

需要：

- 使用低频噪声形成大陆。
- 使用细节噪声轻微扰动。
- 对孤立单格进行清理。
- 去除过小的水池和陆地碎片。
- 在海洋和陆地之间生成浅水与沙滩层。

11. 生态群系系统

11.1 群系判定

群系由以下参数共同决定：

海拔
温度
湿度
离海距离
局部随机扰动
特殊区域权重

11.2 第一版群系

群系	温度	湿度	主要内容
平原	中	中	草地、花、少量树木
森林	中	高	密集树木、蘑菇、动物
沙漠	高	低	沙地、仙人掌、遗迹
雪原	低	中	雪地、松树、冰湖
沼泽	中高	很高	浅水、枯树、毒性植物
山地	任意	任意	石地、矿物、稀疏植物
海岸	任意	任意	沙滩、贝壳、漂流物
海洋	任意	任意	水面、岛屿、海上遗迹

11.3 群系混合

群系边界不能硬切换。

需要设计过渡地带：

森林 → 稀疏森林 → 平原
平原 → 干草地 → 沙漠
平原 → 冻土 → 雪原
陆地 → 沙滩 → 浅海 → 深海

11.4 特殊子群系

后续可以在主群系中生成小型特殊区域：

- 花海

- 枯萎森林
- 红色沙漠
- 冰封湖
- 巨树森林
- 毒沼
- 陨石坑

第一版只需要预留接口。

12. 河流与湖泊

12.1 河流目标

河流应满足：

- 尽量从高处向低处流。
- 最终连接海洋、湖泊或低洼区域。
- 不在区块边缘突然中断。
- 能跨越多个区块保持连续。
- 附近可以生成道路、桥梁和村庄。

12.2 推荐实现

第一版不需要复杂水文模拟。

采用“全局河流场 + 局部栅格化”：

1. 基于低频噪声生成潜在河流线。
2. 使用另一个噪声控制河流宽度。
3. 将河流线映射到具体地块。
4. 在区块边缘使用相同世界坐标计算。
5. 对河岸生成泥地、沙地或湿地。

12.3 湖泊

湖泊可以使用：

- 局部低海拔区域。
- 圆形或不规则噪声遮罩。
- 小型连通区域清理。
- 群系特定湖泊规则。

例如：

雪原：冰湖
沼泽：大量浅水池

沙漠：极少量绿洲
森林：普通淡水湖

13. 地表资源生成

13.1 资源类别

类别	示例
植物	草、花、浆果、药草
树木	普通树、松树、枯树
矿物	石头、煤、铜、铁
食物	浆果、蘑菇、野生作物
特殊资源	水晶、古代碎片、Boss 材料

13.2 分布方法

使用泊松圆盘采样或带最小距离的随机采样。

作用：

- 防止树木全部挤在一起。
- 保持自然随机感。
- 控制资源密度。
- 避免建筑入口被堵住。

13.3 生成规则

每种资源需要数据化配置：

```
ResourceSpawnRule
- resource_id
- allowed_biomes
- min_elevation
- max_elevation
- density
- min_spacing
- cluster_size
- rarity
- required_ground
- forbidden_near_structures
```

13.4 资源持久化

基础世界只记录资源的默认生成状态。

玩家采集后，只保存变化：

该资源节点已被移除
该资源节点剩余耐久
该资源节点下次重生时间

不能保存整个区块所有未变化资源。

14. 建筑与遗迹生成

14.1 第一版建筑类型

类型	规模	功能
废弃营地	小	宝箱、火堆、敌人
小屋	小	NPC、基础物资
遗迹	中	战斗、谜题、宝箱
村庄	大	多建筑、道路、NPC
地牢入口	中	进入地下副本
Boss 祭坛	中	区域 Boss

14.2 建筑模板

建筑应使用模板而不是逐格随机。

每个模板包含：

地面层
墙体层
装饰层
碰撞层
交互点
敌人出生点
宝箱位置
门和入口
道路连接点
允许旋转和镜像

14.3 建筑放置条件

放置前必须检查：

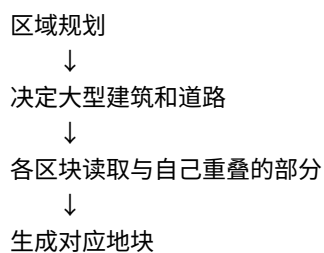
- 地形是否足够平坦。
- 是否与水面重叠。
- 是否与其他建筑重叠。
- 是否距离玩家出生点太近。
- 是否与区块边缘冲突。
- 是否能连接道路。
- 是否符合群系限制。
- 是否保留安全缓冲区域。

14.4 跨区块建筑

大型村庄可能跨越多个区块。

必须由“区域级结构规划器”统一决定位置，不能让每个区块独立生成大型建筑。

推荐结构分两级：



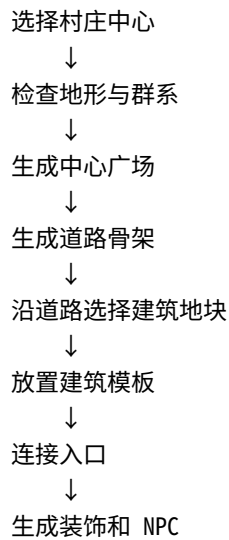
15. 村庄生成

15.1 第一版村庄结构

一个村庄包括：

- 中心广场。
- 3~8 栋建筑。
- 主道路。
- 建筑间小路。
- 水井或火堆。
- 少量 NPC。
- 村庄边界。
- 附近资源区域。

15.2 村庄生成流程



15.3 Wave Function Collapse

第一版不要求完整使用 WFC。

建议：

- 普通房屋使用模板随机。
- 道路和围墙使用连接规则。
- 后续再使用 WFC 生成复杂聚落。

WFC 应作为增强功能，而不是第一阶段的基础依赖。

16. 道路系统

16.1 道路用途

道路负责连接：

- 村庄与村庄。
- 村庄与遗迹。
- 村庄与地牢入口。
- 建筑入口与主道路。
- 重要地点与出生区域。

16.2 路径生成

使用 A*，但道路代价不同于角色寻路。

示例代价：

地形	道路代价
平原	1
森林	3
沙漠	2
沼泽	6
山地	8
浅水	10
深水	不可通行

道路会优先：

- 穿过平坦区域。
- 避开陡峭山地。
- 减少过河次数。
- 使用已有道路。
- 靠近建筑入口。

16.3 桥梁

当道路必须穿越窄河时：

1. 检测河流宽度。
2. 选择较窄的位置。
3. 放置桥梁模板。
4. 标记为道路可通行地块。

第一版只支持短桥。

17. 地下洞穴系统

17.1 地下层结构

地下世界独立于地表，但使用相同区块坐标。

入口对应关系：

地表入口坐标
↓
地下出生点

17.2 洞穴生成方法

第一版使用元胞自动机。

生成步骤：

1. 随机填充墙体。
2. 多次执行邻居规则。
3. 清理过小空洞。
4. 保留主要连通区域。
5. 使用隧道连接分离区域。
6. 放置矿物、敌人和宝箱。
7. 保证入口附近可通行。

17.3 元胞自动机规则示例

周围墙体数量 ≥ 5
当前格变为墙体

周围墙体数量 < 5
当前格变为空地

具体规则需要通过测试调整。

17.4 洞穴类型

第一版：

- 普通石洞
- 地下矿洞
- 地下遗迹

后续增加：

- 冰洞
- 熔岩洞
- 水晶洞
- 真菌洞
- 地底城市

17.5 连通性验证

生成后必须验证：

- 入口不是封闭的。
- 至少存在一个主要探索区域。
- 关键宝箱可到达。
- Boss 房间可到达。
- 不存在必须穿墙才能继续的区域。

18. 玩家系统

18.1 玩家状态

```
IDLE  
WALKING  
RUNNING  
ATTACKING  
USING_TOOL  
INTERACTING  
HURT  
DODGING  
DEAD
```

18.2 玩家属性

```
PlayerStats  
- max_health  
- current_health  
- max_stamina  
- current_stamina  
- move_speed  
- run_speed  
- attack_power  
- defense  
- interaction_range
```

18.3 移动

第一版实现：

- 八方向移动。
- 行走和奔跑。
- 地形减速。
- 碰撞处理。
- 受击击退。
- 简单翻滚或闪避。

移动必须基于物理时间，而不是帧数。

18.4 出生系统

出生点必须满足：

- 位于陆地。

- 不位于障碍内部。
- 附近有基础资源。
- 距离危险建筑足够远。
- 附近至少存在一个安全探索方向。
- 不位于过于极端的群系。

19. 交互和采集

19.1 交互对象

统一接口：

```
Interactable
- can_interact(player)
- get_interaction_text()
- interact(player)
```

可交互对象包括：

- 树木
- 石头
- 植物
- 宝箱
- 门
- NPC
- 地牢入口
- 工作台
- 火堆

19.2 工具系统

第一版工具：

工具	用途
斧头	砍树
镐	采石和采矿
剑	战斗
铲子	挖掘特定地块
火把	地下照明

19.3 资源耐久

每个资源节点拥有：

最大耐久
当前耐久
要求工具类型
最低工具等级
掉落表
受击动画
破坏效果

20. 物品和背包

20.1 物品数据

所有物品使用数据驱动结构：

```
ItemDefinition
- id
- display_name
- description
- icon
- category
- max_stack
- value
- rarity
- usable
- consumable
- equip_slot
- world_scene
```

20.2 物品类型

类型	示例
材料	木材、石头、矿石
工具	斧头、镐
武器	木剑、铁剑
食物	浆果、烤肉
消耗品	药水、火把
装备	护甲、饰品
关键物品	地牢钥匙、Boss 碎片

20.3 背包功能

第一版：

- 固定格子背包。
- 物品堆叠。
- 拖拽交换。
- 拆分堆叠。
- 丢弃物品。
- 快捷栏。
- 自动拾取。
- 容量已满提示。
- 简单排序。

21. 制作系统

21.1 制作配方

```
CraftingRecipe
- id
- ingredients
- output_item
- output_amount
- required_station
- unlock_condition
- crafting_time
```

21.2 第一版制作内容

- 木板
- 木棍
- 木斧
- 木镐
- 石斧
- 石镐
- 木剑
- 火把
- 工作台
- 火堆
- 基础药品

21.3 制作工作站

工作站	功能
徒手制作	最基础物品

工作站	功能
工作台	工具和武器
火堆	烹饪和恢复
熔炉	后续加入矿石冶炼

22. 战斗系统

22.1 第一版战斗方式

采用简单实时近战：

- 普通攻击。
- 连续攻击。
- 攻击冷却。
- 攻击范围。
- 受击。
- 击退。
- 闪避。
- 死亡。
- 掉落。

22.2 伤害公式

第一版使用简单公式：

实际伤害 =
 $\max(1, \text{攻击力} - \text{防御力} \times \text{防御系数})$

后续再增加：

- 暴击
- 元素属性
- 护甲穿透
- 状态效果

22.3 武器数据

WeaponDefinition
- damage
- attack_speed
- range
- knockback
- stamina_cost

- combo_count
- hitbox_scene

22.4 攻击判定

不能依靠敌人与角色中心点距离简单判断。

应使用：

- Area2D 攻击范围。
- 短时间激活碰撞框。
- 每次攻击对同一目标只造成一次伤害。
- 攻击方向跟随玩家朝向。

23. 敌人系统

23.1 第一版敌人

敌人	群系	行为
史莱姆	平原、森林	靠近后跳跃攻击
野狼	森林、雪原	追踪、快速扑击
沙虫	沙漠	潜伏后突袭
沼泽怪	沼泽	缓慢追击、毒性攻击
洞穴蝙蝠	地下	飞行追踪

23.2 敌人状态机

IDLE
WANDER
ALERT
CHASE
ATTACK
HURT
RETURN
DEAD

23.3 敌人生成

敌人生成规则考虑：

- 当前群系。
- 昼夜。

- 距离玩家。
- 区块危险度。
- 已有敌人数量。
- 建筑类型。
- 玩家等级或进度。

23.4 对象池

敌人、掉落物和伤害数字应使用对象池，减少频繁创建和销毁节点。

24. Boss 与区域进度

24.1 第一版 Boss

第一版只需要一个完整 Boss。

示例：

名称：古代守卫
位置：森林遗迹
阶段：三个阶段
奖励：古代核心
作用：解锁地下遗迹区域

24.2 Boss 设计要求

- 有清晰攻击预警。
- 至少三种攻击方式。
- 有阶段变化。
- 不依赖随机秒杀。
- 战斗场地不会被程序化物体堵塞。
- 存在稳定重试机制。
- 击败后永久记录。

24.3 世界进度

世界进度保存：

已击败 Boss
已解锁配方
已发现群系
已激活传送点
已完成遗迹

25. 昼夜系统

25.1 时间结构

建议：

现实 20 分钟 = 游戏一天
白天 12 分钟
黄昏 2 分钟
夜晚 5 分钟
黎明 1 分钟

参数需要可配置。

25.2 昼夜影响

- 环境颜色变化。
- 夜间敌人数量增加。
- 部分资源只在特定时间出现。
- NPC 夜间回屋。
- 火把和灯光更加明显。
- 部分 Boss 只能在夜晚激活。

25.3 时间保存

退出游戏后默认暂停世界时间。

第一版不需要离线时间推进。

26. 天气系统

26.1 第一版天气

天气	适用区域	效果
晴天	全部	默认状态
下雨	森林、平原、沼泽	雨滴、环境变暗
下雪	雪原	雪花、轻微积雪效果
沙尘	沙漠	降低视野、环境粒子

第一版可以只实现晴、雨、雪。

26.2 天气区域化

天气不应瞬间影响整个无限世界。

可按较大的“天气区域”生成：

天气区域大小：8×8 个区块

玩家跨越天气区域时进行平滑过渡。

26.3 天气持久性

天气由：

世界种子
天气区域
游戏日期
时间段

共同决定。

保存时只需要保存当前世界时间，不必保存无限区域的所有天气。

27. 地图与探索

27.1 地图迷雾

玩家未探索区域显示为未知。

区块状态：

未发现
已发现
当前可见

27.2 地图界面

第一版地图显示：

- 已探索地形。
- 玩家位置。
- 出生点。
- 村庄。

- 遗迹。
- 地牢入口。
- 自定义标记。

27.3 探索保存

只保存发现过的区块坐标或压缩位图。

不能保存整个无限地图图像。

28. 世界存档

28.1 存档结构

```
saves/  
├── world_name/  
│   ├── world.json  
│   ├── player.dat  
│   ├── progress.dat  
│   ├── discovered.dat  
│   ├── chunks/  
│   │   ├── surface/  
│   │   └── underground/  
└── backups/
```

28.2 世界元数据

```
{  
  "save_version": 1,  
  "generation_version": 1,  
  "world_name": "My World",  
  "seed": 123456789,  
  "created_at": "...",  
  "last_played_at": "...",  
  "game_time": 12345.0,  
  "player_position": [128, -64],  
  "player_layer": "surface"  
}
```

28.3 差异存档

程序生成世界是确定性的，因此不保存完整基础地图。

只保存玩家造成的变化：

- 被破坏的资源。
- 被放置的方块。
- 被移除的方块。
- 被开启的宝箱。
- 被击败的唯一敌人。
- 建筑门状态。
- NPC 状态。
- 掉落物状态。
- 玩家建造物。

28.4 区块保存条件

区块只有出现永久变化时才创建存档文件。

未修改区块卸载后直接丢弃，下一次根据种子重新生成。

28.5 自动保存

自动保存触发：

- 每隔固定时间。
- 玩家进入新区域。
- 玩家进入地牢。
- 击败 Boss。
- 返回主菜单。
- 游戏正常退出。

保存过程不能明显卡顿。

29. 区块卸载与实体持久化

29.1 实体分类

类型	卸载处理
普通敌人	可直接删除，重新进入时重新生成
唯一敌人	保存击败状态
普通资源	保存采集差异
宝箱	保存开启状态和内容
NPC	保存位置和关键状态
玩家掉落物	按规则保存或超时删除
玩家建筑	永久保存

29.2 睡眠状态

距离玩家较远但尚未卸载的区块：

- 停止敌人 AI。
- 停止动画。
- 降低物理更新。
- 不执行复杂寻路。
- 只保留必要计时。

30. 性能设计

30.1 第一版性能目标

指标	目标
游戏帧率	稳定 60 FPS
最低可接受帧率	30 FPS
活动区块	25 个
预加载区块	49 个
区块尺寸	32×32
同屏普通敌人	30~50
区块生成	平均低于 20 ms
切换区块	无明显卡顿
普通存档体积	低于 100 MB
初始启动时间	低于 5 秒

30.2 性能措施

- 只加载玩家附近区块。
- 使用 TileMapLayer，不为每个地块创建节点。
- 植物和装饰尽量使用 MultiMesh 或批量绘制。
- 敌人和掉落物使用对象池。
- 区块生成分帧执行。
- 复杂生成可以放入 WorkerThreadPool。
- 不在每帧重新计算噪声地图。
- 不在每帧搜索所有世界对象。
- 使用空间分区管理实体。
- 将静态数据存为 Resource。
- 使用脏标记控制存档。

30.3 线程限制

后台线程只能生成纯数据。

后台线程不能直接：

- 创建 Godot 节点。
- 修改 TileMapLayer。
- 操作场景树。
- 修改 UI。

正确流程：

后台线程生成 ChunkData



主线程接收结果



主线程创建地图和节点

31. 调试工具

必须内置开发者调试界面。

31.1 地图调试图层

支持切换显示：

- 海拔噪声。
- 温度噪声。
- 湿度噪声。
- 群系分布。
- 区块边界。
- 区块坐标。
- 河流图。
- 建筑规划区域。
- 道路代价。
- 实体生成点。
- 寻路网格。

31.2 调试命令

第一版支持：

```
teleport x y
set_time hour
set_weather type
```

```
give item amount
spawn enemy
reload_chunk
regenerate_chunk
show_seed
set_biome_debug
kill_all_enemies
save_world
```

31.3 世界生成预览器

单独制作一个开发工具场景：

- 输入种子。
- 生成大范围低分辨率世界预览。
- 调整噪声频率。
- 调整阈值。
- 查看群系分布。
- 导出预览图片。
- 快速跳转到特定坐标。

32. UI 设计

32.1 主菜单

```
继续游戏
创建世界
加载世界
设置
制作人员
退出游戏
```

32.2 创建世界界面

包含：

- 世界名称。
- 世界种子。
- 随机种子按钮。
- 难度。
- 世界大小显示为“无限”。
- 创建按钮。
- 返回按钮。

32.3 游戏 HUD

左上：生命、体力、状态效果
右上：小地图、时间、天气、群系
底部：快捷栏
左下：交互提示
中央：准星或交互标记

32.4 暂停界面

继续游戏
背包
设置
保存游戏
返回主菜单
退出游戏

33. 音效和音乐

33.1 音效类别

- 脚步。
- 砍树。
- 采矿。
- 拾取。
- 攻击。
- 受伤。
- 敌人。
- 开门。
- 宝箱。
- 雨声。
- 风声。
- 洞穴环境声。

33.2 环境音

根据群系和时间切换：

森林：鸟鸣、树叶
沙漠：风声
雪原：寒风
沼泽：虫鸣、水声

洞穴：滴水、回声
夜晚：夜虫和远处敌人

33.3 音乐系统

音乐根据：

- 群系。
- 战斗状态。
- Boss 战。
- 昼夜。
- 地表或地下。

进行淡入淡出切换。

34. 文件目录结构

```
infinite-frontier/  
├── project.godot  
├── README.md  
├── AGENTS.md  
├── LICENSE  
├──  
├── assets/  
│   ├── sprites/  
│   ├── tilesets/  
│   ├── animations/  
│   ├── audio/  
│   ├── fonts/  
│   └── shaders/  
├──  
├── data/  
│   ├── items/  
│   ├── recipes/  
│   ├── biomes/  
│   ├── resources/  
│   ├── enemies/  
│   ├── structures/  
│   └── loot_tables/  
├──  
├── scenes/  
│   ├── main/  
│   ├── world/  
│   ├── player/  
│   ├── enemies/  
│   ├── structures/  
│   ├── interactables/  
│   └── ui/
```

```

|   └── debug/
|
|── scripts/
|   ├── core/
|   |   ├── game_manager.gd
|   |   ├── event_bus.gd
|   |   ├── save_manager.gd
|   |   └── settings_manager.gd
|   |
|   ├── world/
|   |   ├── world_manager.gd
|   |   ├── chunk_manager.gd
|   |   ├── chunk_data.gd
|   |   ├── chunk_renderer.gd
|   |   ├── world_coordinates.gd
|   |   └── world_layer_manager.gd
|   |
|   ├── generation/
|   |   ├── seed_manager.gd
|   |   ├── terrain_generator.gd
|   |   ├── biome_generator.gd
|   |   ├── river_generator.gd
|   |   ├── resource_generator.gd
|   |   ├── structure_generator.gd
|   |   ├── road_generator.gd
|   |   └── cave_generator.gd
|   |
|   ├── player/
|   ├── combat/
|   ├── inventory/
|   ├── crafting/
|   ├── enemies/
|   ├── interaction/
|   ├── time_weather/
|   ├── persistence/
|   └── debug/
|
|── tests/
|   ├── generation/
|   ├── world/
|   ├── persistence/
|   └── gameplay/
|
|── docs/
|   ├── architecture.md
|   ├── world-generation.md
|   ├── save-format.md
|   ├── gameplay-systems.md
|   └── development-log.md
|
|── tools/

```

```
|—— world_preview/  
|—— structure_editor/  
|—— data_validator/
```

35. 开发阶段

阶段一：基础工程与角色移动

目标

建立可运行的 Godot 项目。

功能

- 创建项目目录。
- 创建主菜单。
- 创建游戏主场景。
- 实现角色八方向移动。
- 实现摄像机跟随。
- 实现像素对齐。
- 实现基础碰撞。
- 创建调试信息面板。
- 添加基础测试场景。

验收标准

- 项目可正常启动。
- 玩家移动流畅。
- 摄像机不抖动。
- 画面保持像素清晰。
- 代码目录结构正确。

阶段二：单区块地形生成

目标

使用世界种子生成一个固定区块。

功能

- 实现世界种子管理。
- 实现稳定随机数。
- 实现海拔噪声。
- 生成陆地、水面和沙滩。
- 使用 TileMapLayer 显示地形。
- 显示区块坐标。
- 支持重新生成。

验收标准

- 相同种子结果完全一致。
 - 不同种子明显不同。
 - 海岸不存在大量单格噪点。
 - 生成逻辑与显示逻辑分离。
-

阶段三：无限区块加载

目标

实现真正的动态世界。

功能

- 世界坐标与区块坐标转换。
- 玩家跨区块检测。
- 加载玩家附近区块。
- 卸载远处区块。
- 区块缓存。
- 生成优先级。
- 区块边界调试显示。
- 负坐标支持。

验收标准

- 玩家可以连续移动多个区块。
 - 区块边缘无明显接缝。
 - 返回旧区块时结果一致。
 - 内存不会无限增长。
 - 负坐标区域正常生成。
-

阶段四：生态群系

目标

加入温度、湿度和多种群系。

功能

- 温度噪声。
- 湿度噪声。
- 群系判定。
- 平原、森林、沙漠、雪原、沼泽。
- 群系过渡。
- 群系调试图层。
- 群系配置数据化。

验收标准

- 群系分布具有大片连续区域。
 - 不出现大量随机碎片。
 - 群系过渡自然。
 - 相同坐标群系稳定。
-

阶段五：资源与装饰生成

目标

使世界具有可探索内容。

功能

- 树木、石头、植物。
- 资源密度配置。
- 泊松采样或最小距离生成。
- 群系特定资源。
- 资源碰撞。
- 资源受击和破坏。
- 基础掉落物。
- 简单拾取。

验收标准

- 资源不会重叠。
 - 资源不会生成在水中或建筑入口。
 - 采集后可以获得物品。
 - 返回区块时被采集资源不会错误恢复。
-

阶段六：存档与区块差异

目标

建立稳定的世界持久化。

功能

- 创建世界。
- 保存世界元数据。
- 保存玩家位置。
- 保存被破坏资源。
- 区块差异文件。
- 自动保存。
- 加载世界。
- 存档版本。
- 备份机制。

验收标准

- 退出后玩家位置正确恢复。
 - 被砍掉的树不会重新出现。
 - 未修改区块不创建大量文件。
 - 损坏存档能显示明确错误。
 - 自动保存不会明显卡顿。
-

阶段七：背包、物品与制作

目标

形成基础生存循环。

功能

- 物品数据定义。
- 背包格子。
- 物品堆叠。
- 快捷栏。
- 拾取和丢弃。
- 工具装备。
- 制作配方。
- 工作台。
- 基础工具和武器。

验收标准

- 物品不会复制或丢失。
 - 堆叠数量正确。
 - 配方材料不足时不能制作。
 - 存档后背包正确恢复。
-

阶段八：敌人与战斗

目标

加入基础游戏挑战。

功能

- 玩家攻击。
- 伤害和生命。
- 受击与击退。
- 敌人状态机。
- 敌人追踪。
- 敌人攻击。
- 敌人死亡和掉落。

- 对象池。
- 玩家死亡和重生。

验收标准

- 攻击判定稳定。
 - 敌人不会穿过障碍。
 - 同一次攻击不会重复伤害。
 - 玩家死亡后能正常重生。
 - 敌人数量受限制。
-

阶段九：建筑、村庄与道路

目标

生成有目的性的探索地点。

功能

- 建筑模板。
- 营地。
- 小屋。
- 遗迹。
- 村庄中心。
- 建筑放置检查。
- A* 道路。
- 短桥。
- 宝箱和基础 NPC。

验收标准

- 建筑不会互相重叠。
 - 建筑不会生成在深水中。
 - 建筑入口可到达。
 - 道路能连接主要建筑。
 - 跨区块建筑不会被切断。
-

阶段十：地下洞穴

目标

增加第二个可探索世界层。

功能

- 地牢入口。
- 地表与地下切换。
- 元胞自动机洞穴。

- 洞穴连通性修复。
- 地下矿物。
- 地下敌人。
- 地下宝箱。
- 火把照明。

验收标准

- 洞穴入口可正常往返。
 - 洞穴主要区域连通。
 - 不会出生在墙体中。
 - 关键目标均可到达。
 - 地下区块能正常加载和保存。
-

阶段十一：昼夜与天气

目标

增强世界氛围。

功能

- 世界时间。
- 环境颜色变化。
- 夜间敌人。
- 晴天、雨天和雪天。
- 天气区域。
- 环境音切换。
- 时间和天气保存。

验收标准

- 时间变化平滑。
 - 天气不会突然闪烁切换。
 - 雪只在合理区域出现。
 - 保存加载后时间状态正确。
-

阶段十二：Boss 与完整游戏循环

目标

形成可完成的第一版游戏。

功能

- 森林遗迹。
- Boss 祭坛。
- 一个三阶段 Boss。

- Boss 奖励。
- 配方解锁。
- 世界进度保存。
- 通关提示。
- 基础成就或统计。

验收标准

- 玩家可以从创建世界开始逐步发展。
 - 可以制作装备。
 - 可以发现遗迹。
 - 可以进入 Boss 战。
 - 可以击败 Boss。
 - Boss 击败状态永久保存。
-

阶段十三：优化、打包与文档

目标

形成可公开展示的软件。

功能

- 性能分析。
- 区块生成优化。
- 对象池优化。
- UI 完善。
- 音效和音乐。
- 设置界面。
- 键位修改。
- Windows EXE 打包。
- README。
- 用户指南。
- 演示视频。

验收标准

- 新电脑无需安装 Godot 即可运行。
 - 主要流程没有阻断性错误。
 - 普通电脑稳定运行。
 - 存档兼容。
 - README 包含截图、功能和操作方法。
-

36. 测试计划

36.1 世界生成测试

测试	内容
种子确定性	同一种子和坐标生成相同结果
顺序无关性	不同探索顺序不影响区块内容
边界连续性	相邻区块地形无断裂
负坐标	世界左侧和上方正常生成
群系连续性	不出现大量单格群系
建筑合法性	建筑不重叠、不落入深水
河流连续性	跨区块河流不突然中断
洞穴连通性	入口与主要区域连通

36.2 存档测试

- 创建世界后保存。
- 破坏资源后保存。
- 放置物品后保存。
- 开启宝箱后保存。
- 击败唯一敌人后保存。
- 在地下退出后加载。
- 游戏异常关闭后的恢复。
- 旧版本存档迁移。
- 大量区块存档性能。

36.3 性能测试

测试场景：

连续向一个方向移动 30 分钟
快速传送到远距离坐标
频繁往返区块边界
同屏生成大量敌人
大量采集和放置物品
快速进入和退出地下
连续自动保存

必须监控：

- 帧率。
- 内存。
- 节点数量。

- 区块生成耗时。
 - 存档耗时。
 - TileMapLayer 更新耗时。
 - 活动敌人数量。
-

37. 典型错误与防范

37.1 区块边缘接缝

原因：

- 每个区块独立使用局部坐标计算噪声。
- 不同区块随机序列不一致。

解决：

- 所有噪声使用世界绝对坐标。
- 坐标随机必须基于世界坐标和派生种子。

37.2 探索顺序改变世界

原因：

- 使用一个不断推进的全局随机数生成器。

解决：

- 每个区块和生成模块使用独立局部种子。

37.3 内存持续增长

原因：

- 区块节点未真正释放。
- 信号连接保留引用。
- 缓存没有大小上限。
- 敌人和掉落物未卸载。

解决：

- 为缓存设置上限。
- 区块卸载时清理节点和信号。
- 使用性能监视器长期测试。

37.4 存档体积过大

原因：

- 保存完整生成地图。
- 保存所有未变化对象。

解决：

- 只保存与基础生成结果不同的内容。

37.5 地图生成卡顿

原因：

- 同一帧生成大量区块。
- 主线程计算复杂建筑和洞穴。

解决：

- 使用生成队列。
- 分帧处理。
- 后台线程生成纯数据。
- 优先加载玩家前进方向。

37.6 大型建筑被区块切断

原因：

- 区块独立决定大型建筑。

解决：

- 使用区域级结构规划器。
- 区块只读取与自己重叠的结构数据。

38. Codex 开发规则

仓库根目录创建 `AGENTS.md`：

1. 每次修改前阅读 README.md、AGENTS.md 和相关模块。
2. 不允许一次性实现整个项目，只完成当前阶段。
3. 每个阶段结束时项目必须可启动、可运行。
4. 世界生成必须具有确定性。
5. 不允许使用依赖生成顺序的全局随机数。
6. 地图生成逻辑不得直接依赖玩家节点。
7. 存档系统只保存世界差异，不保存完整基础地图。
8. 不允许为每个地块创建独立 Node2D。

9. 后台线程不得修改场景树。
10. 新增核心生成算法时必须增加测试或验证场景。
11. 所有静态游戏内容优先使用数据驱动方式。
12. 不允许在 UI 脚本中实现世界生成和战斗逻辑。
13. 不允许吞掉异常或只打印模糊错误。
14. 修改存档格式时必须更新 `save_version`。
15. 修改世界生成算法时必须更新 `generation_version`。
16. 必须支持负世界坐标。
17. 必须测试区块边缘和跨区块内容。
18. 所有资源 ID 必须稳定且唯一。
19. 不允许为了演示写死区块内容。
20. 每完成一个阶段，更新 README 和开发日志。

39. Codex 任务执行顺序

不要直接把整份计划要求 Codex 一次实现。

推荐依次下达：

- 任务 1：初始化 Godot 项目和目录结构
- 任务 2：实现角色移动、摄像机和像素对齐
- 任务 3：实现世界种子和稳定派生种子
- 任务 4：实现单区块基础地形生成
- 任务 5：实现世界坐标和区块坐标转换
- 任务 6：实现区块动态加载和卸载
- 任务 7：实现多噪声地形和海岸
- 任务 8：实现温度、湿度和群系
- 任务 9：实现资源配置和程序化分布
- 任务 10：实现资源交互和采集
- 任务 11：实现世界元数据和玩家存档
- 任务 12：实现区块差异存档
- 任务 13：实现物品、背包和快捷栏
- 任务 14：实现工具和制作系统
- 任务 15：实现玩家战斗
- 任务 16：实现敌人状态机和对象池
- 任务 17：实现建筑模板系统
- 任务 18：实现遗迹、营地和宝箱
- 任务 19：实现村庄布局和道路
- 任务 20：实现地表与地下世界切换
- 任务 21：实现元胞自动机洞穴
- 任务 22：实现昼夜循环
- 任务 23：实现天气系统
- 任务 24：实现 Boss 和世界进度
- 任务 25：制作调试工具和世界预览器
- 任务 26：性能优化和长期运行测试

任务 27: 完善 UI、音效和设置
任务 28: 打包 Windows EXE 和编写文档

每个任务都必须包含：

目标
允许修改的模块
具体实现内容
禁止修改的内容
测试方法
验收标准
已知风险

40. 第一版完整演示流程

最终演示世界应呈现以下过程：

1. 玩家创建名为“无尽边境”的世界。
2. 输入固定世界种子。
3. 出生在森林与平原交界处。
4. 玩家向东探索，地图自动加载新区块。
5. 发现河流和跨河木桥。
6. 采集木材和石头。
7. 制作斧头、镐和木剑。
8. 夜晚出现敌人。
9. 玩家击败敌人并获得材料。
10. 发现一个程序生成的小型村庄。
11. 沿道路找到森林遗迹。
12. 打开遗迹宝箱，获得地下钥匙。
13. 进入程序生成的地下洞穴。
14. 采集矿石并击败地下敌人。
15. 返回地表制作更强装备。
16. 到达 Boss 祭坛。
17. 击败区域 Boss。
18. 保存并退出游戏。
19. 重新进入后位置、背包、资源和 Boss 状态全部正确恢复。
20. 继续向远方探索，生成新的未知区域。

41. 最小可玩版本

不要一开始就追求完整第一版。

最小可玩版本只需要：

模块	内容
世界	无限区块、平原和森林
玩家	移动、生命、交互
资源	树木、石头
物品	木材、石头、斧头、镐
背包	基础背包和快捷栏
制作	斧头、镐、木剑
战斗	一种敌人
存档	玩家、背包、资源差异
建筑	一个遗迹模板
目标	找到遗迹并打开宝箱

最小可玩版本完成后，再依次加入：

更多群系
 ↓
 村庄和道路
 ↓
 洞穴
 ↓
 昼夜与天气
 ↓
 Boss
 ↓
 完整内容

42. 最终交付物

交付物	内容
Windows 游戏	可直接运行的 EXE
Godot 工程	完整源代码
示例世界	一个固定种子的展示世界
世界预览器	查看种子与群系分布
结构编辑器	制作建筑模板
README	项目介绍和运行方法
架构文档	世界、区块和存档架构

交付物	内容
生成算法文档	噪声、群系、洞穴和道路
存档格式文档	世界元数据和差异数据
测试报告	确定性、性能和存档测试
演示视频	完整探索和 Boss 流程
学习总结	问题发现、定位和解决过程

43. 第一版完成标准

只有满足以下条件才算完成：

- 相同世界种子能够稳定生成相同世界。
- 玩家可以向任意方向持续探索。
- 区块可以动态加载和卸载。
- 区块之间没有明显地形断层。
- 至少存在五种生态群系。
- 至少存在三种程序生成建筑。
- 存在地表和地下两个世界层。
- 玩家可以采集、制作和战斗。
- 至少存在三种普通敌人。
- 至少存在一个完整 Boss。
- 昼夜和天气能够正常变化。
- 玩家修改过の世界能够永久保存。
- 未修改区块不会产生大量存档。
- 运行过程中内存不会持续无上限增长。
- 可以打包为 Windows 独立程序。
- 不需要网络、API 或任何 AI 模型。